



华南理工大学

South China University of Technology

本科毕业设计（论文）

代码覆盖率及路径分析检测工具

学 院	计算机学院
专 业	软件工程
学 生 姓 名	许 同 学
学 生 学 号	2023000000
指 导 教 师	张老师 (副教授)
提 交 日 期	2026 年 4 月 2 日

华南理工大学

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名： 将这里替换成
您的签名文件



日期：2026 年 4 月 2 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保存并向国家有关部门或机构送交论文的复印件和电子版，允许学位论文被查阅；学校可以公布学位论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存、汇编学位论文。本人电子文档的内容和纸质论文的内容相一致。

作者签名： 将这里替换成
您的签名文件



日期：2026 年 4 月 2 日

指导教师签名：

日期：2026 年 4 月 2 日

作者联系电话：13800138000

电子邮箱：student@university.edu

摘 要

本文设计并实现了一个 Python 程序执行路径追踪和代码覆盖率分析工具——Path-Tracer。该工具利用 Python 标准库中的 `sys.settrace` 机制实现运行时追踪，记录程序执行过程中经过的每一行代码；同时使用抽象语法树（AST）进行静态分析，识别源代码中的所有控制流分支点（包括 if 条件语句、for 循环、while 循环、try/except 异常处理等）。通过对比静态分析得到的分支点与运行时追踪的执行路径，计算代码覆盖率，并生成多种格式的 report（文本、JSON、HTML）。实验结果表明，该工具能够有效分析 Python 程序的基本结构流，包括顺序、选择和循环结构，准确记录执行路径并生成详细的分支覆盖报告。

关键词： 代码覆盖率；执行路径；静态分析；动态追踪；控制流图

Abstract

This paper designs and implements a Python execution path tracing and code coverage analysis tool called PathTracer. The tool uses Python's standard library `sys.settrace` mechanism for runtime tracing, recording each line of code executed during program execution. It also performs static analysis using Abstract Syntax Tree (AST) to identify all control flow branch points in source code, including if statements, for loops, while loops, and try/except handlers. By comparing the branch points from static analysis with the execution paths from runtime tracing, the tool calculates code coverage and generates reports in multiple formats (text, JSON, HTML). Experimental results demonstrate that the tool effectively analyzes basic program structures in Python, including sequential, selection, and loop structures, accurately records execution paths, and generates detailed branch coverage reports.

Keywords: Code Coverage; Execution Path; Static Analysis; Dynamic Tracing; Control Flow Graph

目 录

摘 要	I
Abstract	II
目 录	III
插图目录	VII
表格目录	VIII
第一章 实验目的和背景	1
1.1 实验目的	1
1.2 实验背景	1
1.2.1 代码覆盖率概述	1
1.2.2 Python 追踪机制	1
1.2.3 AST 静态分析	2
1.3 作业要求	2
第二章 系统设计与实现	3
2.1 系统架构	3
2.1.1 模块划分	3
2.2 核心数据模型	3
2.2.1 分支类型枚举	3
2.2.2 代码分支数据类	4
2.2.3 函数调用记录类	4
2.2.4 覆盖率报告类	5
2.3 运行时追踪器实现	5
2.3.1 sys.settrace 机制原理	5
2.3.2 追踪回调函数核心实现	5
2.3.3 参数提取实现	7
2.3.4 上下文管理器接口	8
2.4 控制流图构建器	9

2.4.1	AST 遍历原理.....	9
2.4.2	分支节点识别.....	9
2.4.3	结束行号计算.....	11
2.5	报告生成器.....	12
2.5.1	覆盖率计算核心逻辑.....	12
2.5.2	覆盖率计算公式.....	13
2.5.3	多格式报告输出.....	13
2.6	HTML 报告生成.....	14
2.6.1	HTML 报告结构.....	14
2.6.2	执行状态视觉区分.....	14
2.7	本章小结.....	14
第三章	实验过程与结果.....	15
3.1	实验环境.....	15
3.2	测试目标程序.....	15
3.2.1	程序简介.....	15
3.2.2	程序结构分析.....	15
3.3	测试用例设计与执行.....	16
3.3.1	测试用例设计.....	16
3.3.2	测试执行过程.....	16
3.4	实验结果.....	17
3.4.1	覆盖率统计汇总.....	17
3.4.2	测试套件累计覆盖率.....	17
3.4.3	TC2 详细分支覆盖分析.....	17
3.4.4	未覆盖分支分类分析.....	19
3.5	Demo 程序完整测试.....	19
3.5.1	Demo 程序结构.....	19
3.5.2	Demo 程序覆盖率结果.....	20
3.5.3	分支类型分布.....	20
3.5.4	函数调用树.....	20
3.6	与 coverage.py 对比分析.....	21
3.6.1	对比口径说明.....	21
3.6.2	对比实验设计.....	21

3.6.3	覆盖率对比结果	22
3.6.4	功能对比分析	22
3.6.5	对比结论	22
3.7	HTML 报告展示	22
3.8	结果分析与讨论	23
3.8.1	覆盖率分析结论	23
3.8.2	工具优势	23
3.8.3	工具局限性	23
3.8.4	覆盖率合理性说明	24
3.8.5	实验对象说明	24
3.9	HTML 报告与可视化	24
3.9.1	HTML 报告特性	24
3.9.2	控制流图示例	24
3.10	本章小结	25
第四章	使用文档	26
4.1	安装与配置	26
4.1.1	环境要求	26
4.1.2	获取源码	26
4.2	命令行使用	26
4.2.1	基本用法	26
4.2.2	命令行选项	26
4.2.3	使用示例	26
4.3	API 使用	27
4.3.1	基本追踪流程	27
4.3.2	获取函数调用树	27
4.4	报告格式说明	28
4.4.1	文本格式	28
4.4.2	JSON 格式	28
4.4.3	HTML 格式	29
4.5	支持的程序结构	29
4.6	本章小结	29

结论	30
参考文献	31
附录 A 核心源代码	33
A.1 数据模型 (models.py)	33
A.2 运行时追踪器 (tracer.py)	33
A.3 控制流图构建器 (cfg_builder.py)	33
A.4 报告生成器 (reporter.py)	33
A.5 命令行接口 (cli.py)	33

插图目录

2-1 PathTracer 系统架构图	3
----------------------------	---

表格目录

2-1	模块职责划分	3
2-2	追踪事件类型	5
2-3	报告输出格式	13
2-4	代码行状态样式	14
3-1	实验环境配置	15
3-2	目标程序结构	16
3-3	测试用例设计	16
3-4	测试用例覆盖率对比	17
3-5	测试套件累计覆盖率分析	17
3-6	TC2 已执行分支详情	18
3-7	未覆盖分支分类及原因	19
3-8	Demo 程序覆盖率统计	20
3-9	Demo 程序分支类型分布	20
3-10	覆盖率统计口径对比	21
3-11	PathTracer vs coverage.py 覆盖率对比 (demo.py)	22
3-12	功能特性对比	22
3-13	工具局限性及影响	23
3-14	HTML 报告可视化元素	24
4-1	命令行选项说明	26
4-2	支持的程序结构	29

第一章 实验目的和背景

1.1 实验目的

本次课程作业的主要目的是设计并实现一个能够记录 Python 程序执行路径和覆盖率的工具。具体目标包括：

- 1) 掌握 Python 运行时追踪技术，理解 `sys.settrace` 的工作原理
- 2) 学习使用抽象语法树（AST）进行静态代码分析
- 3) 能够识别程序中的基本控制流结构：顺序、选择（if/elif/else）和循环（for/while）
- 4) 记录程序运行时的执行路径，包括函数调用关系
- 5) 计算代码覆盖率并生成可读的执行分支报告

1.2 实验背景

1.2.1 代码覆盖率概述

代码覆盖率是软件测试中的重要指标，用于衡量测试用例对被测代码的覆盖程度。常见的覆盖率类型包括：

- **语句覆盖**：测量被执行的代码语句比例
- **分支覆盖**：测量控制流分支的执行比例（如 if 语句的 true/false 分支）
- **路径覆盖**：测量程序执行路径的覆盖比例

本实验主要关注**分支覆盖**，即识别程序中的控制流分支点，并统计在运行时被触发的分支数量。

1.2.2 Python 追踪机制

Python 提供了 `sys.settrace()` 函数，允许注册一个回调函数来监控程序的执行。当程序执行时，Python 解释器会在以下事件发生时调用回调函数：

- `'call'`：函数被调用时
- `'line'`：执行新的一行代码时
- `'return'`：函数返回时
- `'exception'`：发生异常时

利用这一机制，可以精确记录程序的执行路径。

1.2.3 AST 静态分析

Python 的 `ast` 模块可以将源代码解析为抽象语法树 (AST)。通过遍历 AST，可以识别程序中的所有控制流结构：

- `ast.If`: if 条件语句
- `ast.For`: for 循环语句
- `ast.While`: while 循环语句
- `ast.ExceptHandler`: 异常处理器

1.3 作业要求

根据作业要求，本工具需要满足以下功能：

- 1) 记录 Python 程序执行路径和覆盖率
- 2) 分析基本程序常见结构流：选择、顺序与循环
- 3) 记录程序运行执行路径
- 4) 生成描述程序运行执行分支的报告

程序结构需要清晰，代码需要有必要的注释，并提供使用文档。

第二章 系统设计与实现

2.1 系统架构

PathTracer 系统采用模块化设计，由以下核心组件构成：

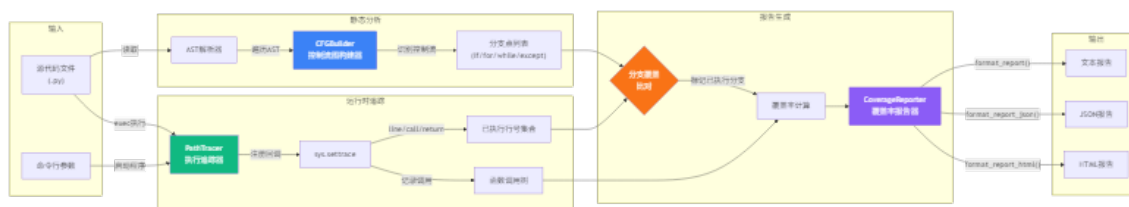


图 2-1 PathTracer 系统架构图

2.1.1 模块划分

系统主要包含以下模块：

表 2-1 模块职责划分

模块	文件	核心职责
数据模型	models.py	定义分支类型枚举、代码分支、函数调用、执行路径、覆盖率报告等数据结构
运行时追踪	tracer.py	使用 <code>sys.settrace</code> 追踪程序执行，记录行号和函数调用
控制流分析	cfg_builder.py	使用 AST 静态分析识别源代码中的所有分支点
报告生成	reporter.py	对比静态分支与执行路径，计算覆盖率，生成多格式报告
命令行接口	cli.py	解析命令行参数，协调各模块工作流程

2.2 核心数据模型

2.2.1 分支类型枚举

定义了程序中可能出现的控制流分支类型：

Listing 2.1 分支类型枚举定义

```
class BranchType(Enum):  
    """ 控制流分支类型枚举 """
```

```

SEQUENTIAL = "sequential"  # 顺序执行
IF = "if"                  # if 条件语句
ELSE = "else"              # else 分支
ELIF = "elif"              # elif 分支
FOR = "for"                 # for 循环
WHILE = "while"             # while 循环
TRY = "try"                 # try 异常捕获块
EXCEPT = "except"         # except 异常处理块

```

2.2.2 代码分支数据类

CodeBranch 表示源代码中的一个分支点:

Listing 2.2 CodeBranch 数据类

```

@dataclass
class CodeBranch:
    """ 表示代码中的一个分支点 """
    file_path: str          # 源文件路径
    line_no: int             # 分支起始行号
    branch_type: BranchType # 分支类型
    end_line: int = 0        # 分支结束行号
    is_executed: bool = False # 运行时是否被执行

```

2.2.3 函数调用记录类

FunctionCall 记录函数调用信息, 支持嵌套调用树:

Listing 2.3 FunctionCall 数据类

```

@dataclass
class FunctionCall:
    """ 记录函数调用信息 """
    function_name: str      # 函数名
    file_path: str          # 所在文件
    line_no: int            # 调用行号
    call_depth: int         # 调用深度 (0 为顶层)

```

```
arguments: Dict[str, str]          # 参数名到值的映射
return_value: Optional[str]       # 返回值 (repr后的字符串)
children: List['FunctionCall']    # 子调用列表
```

2.2.4 覆盖率报告类

CoverageReport 汇总分析结果:

Listing 2.4 CoverageReport 数据类

```
@dataclass
class CoverageReport:
    """ 覆盖率分析报告 """
    total_branches: int = 0          # 总分支数
    executed_branches: int = 0       # 已执行分支数
    branches_by_type: Dict[BranchType, int] # 按类型统计
    coverage_percentage: float = 0.0 # 覆盖率百分比
    file_reports: Dict[str, ExecutionPath] # 各文件详细报告
```

2.3 运行时追踪器实现

2.3.1 sys.settrace 机制原理

Python 的 `sys.settrace` 函数允许注册一个全局追踪回调。当程序执行时，解释器会在每个“事件”发生时调用该回调：

表 2-2 追踪事件类型

事件	触发时机
'call'	函数被调用时（进入函数体前）
'line'	即将执行新的一行代码时
'return'	函数即将返回时（ <code>arg</code> 为返回值）
'exception'	发生异常时（ <code>arg</code> 为异常元组）

2.3.2 追踪回调函数核心实现

Listing 2.5 追踪回调函数实现

```
def _trace_callback(self, frame, event: str, arg):
```

""" 内部追踪回调函数

Args:

frame: 当前栈帧对象, 包含代码对象、局部变量等

event: 事件类型 ('call', 'line', 'return', 'exception')

arg: 事件相关参数 (return 时为返回值, exception 时为异常信息)

Returns:

返回自身以继续追踪, 或None停止追踪

"""

```
if not self._target_file:
```

```
    return None
```

```
code = frame.f_code
```

```
# 只追踪目标文件
```

```
if not code.co_filename.endswith(self._target_file):
```

```
    return self._trace_callback
```

```
if event == 'line':
```

```
    # 记录执行的行号
```

```
    line_no = frame.f_lineno
```

```
    self.execution_paths[self._target_file] \
```

```
        .executed_lines.add(line_no)
```

```
elif event == 'call':
```

```
    # 提取函数名和参数
```

```
    func_name = code.co_name
```

```
    args = self._extract_arguments(frame, code)
```

```
    # 构建函数调用对象
```

```
    call_depth = len(self._call_stack)
```

```
    func_call = FunctionCall(
```

```
        function_name=func_name,
```



```

        file_path=code.co_filename ,
        line_no=frame.f_lineno ,
        call_depth=call_depth ,
        arguments=args
    )
    # 添加到调用树
    if self._call_stack:
        self._call_stack[-1].children.append(func_call)
    else:
        self.function_calls.append(func_call)
    self._call_stack.append(func_call)

elif event == 'return':
    # 弹出调用栈，记录返回值
    if self._call_stack:
        func_call = self._call_stack.pop()
        if arg is not None:
            func_call.return_value = repr(arg)

return self._trace_callback

```

2.3.3 参数提取实现

Listing 2.6 函数参数提取

```

def _extract_arguments(self, frame, code) -> Dict[str, str]:
    """从栈帧中提取函数参数

    利用code对象获取参数名列表，从frame.f_locals获取参数值
    """
    args = {}
    try:
        arg_count = code.co_argcount
        arg_names = code.co_varnames[:arg_count]

```

```

    for arg_name in arg_names:
        if arg_name in frame.f_locals:
            try:
                args[arg_name] = repr(frame.f_locals[arg_name])
            except Exception:
                args[arg_name] = '<unable to repr>'
    except Exception:
        pass
    return args

```

2.3.4 上下文管理器接口

使用 Python 的上下文管理器协议简化追踪的启动和停止：

Listing 2.7 上下文管理器实现

```

def __enter__(self):
    """ 进入上下文：保存原追踪函数，启动追踪 """
    self._original_trace = sys.gettrace()
    sys.settrace(self._trace_callback)
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    """ 退出上下文：恢复原追踪函数 """
    sys.settrace(self._original_trace)
    return False # 不抑制异常

```

使用示例：

```

tracer = PathTracer().trace_file("target.py")
with tracer:
    # 在此执行目标代码
    result = target_function()
# 退出 with 块后自动停止追踪

```

2.4 控制流图构建器

2.4.1 AST 遍历原理

Python 的 `ast` 模块可以将源代码解析为抽象语法树。通过遍历 AST，可以识别所有控制流节点：

Listing 2.8 AST 遍历构建 CFG

```
def build_from_file(self, file_path: str) -> List[CodeBranch]:
    """从Python文件构建控制流图

    Args:
        file_path: Python源文件路径

    Returns:
        识别到的所有分支点列表
    """
    with open(file_path, 'r', encoding='utf-8') as f:
        source = f.read()

    # 解析源代码为AST
    tree = ast.parse(source, filename=file_path)
    self.branches = []

    # 遍历AST, 提取控制流节点
    for node in ast.walk(tree):
        branch = self._extract_branch(file_path, node)
        if branch:
            self.branches.append(branch)

    return self.branches
```

2.4.2 分支节点识别

Listing 2.9 分支节点识别逻辑

```
def _extract_branch(self, file_path: str, node: ast.AST):
    """从AST节点提取分支信息

    识别 if/for/while/except 等控制流结构
    """
    # if 条件语句
    if isinstance(node, ast.If):
        return CodeBranch(
            file_path=file_path,
            line_no=node.lineno,
            branch_type=BranchType.IF,
            end_line=self._get_end_line(node)
        )

    # for 循环
    elif isinstance(node, ast.For):
        return CodeBranch(
            file_path=file_path,
            line_no=node.lineno,
            branch_type=BranchType.FOR,
            end_line=self._get_end_line(node)
        )

    # while 循环
    elif isinstance(node, ast.While):
        return CodeBranch(
            file_path=file_path,
            line_no=node.lineno,
            branch_type=BranchType.WHILE,
            end_line=self._get_end_line(node)
        )
```

```

# except 异常处理器
elif isinstance(node, ast.ExceptHandler):
    return CodeBranch(
        file_path=file_path,
        line_no=node.lineno,
        branch_type=BranchType.EXCEPT,
        end_line=self._get_end_line(node)
    )

return None

```

2.4.3 结束行号计算

Listing 2.10 结束行号计算

```

def _get_end_line(self, node: ast.AST) -> int:
    """ 获取代码块的结束行号

    优先使用 Python 3.8+ 的 end_lineno 属性,
    否则递归计算 body 中最大的行号
    """
    # Python 3.8+ 提供 end_lineno 属性
    if hasattr(node, 'end_lineno') and node.end_lineno is not None:
        return node.end_lineno

    # 递归计算 body 中的最大行号
    if hasattr(node, 'body') and node.body:
        return max(self._get_end_line(n) for n in node.body)

    return getattr(node, 'lineno', 0)

```

2.5 报告生成器

2.5.1 覆盖率计算核心逻辑

Listing 2.11 覆盖率计算实现

```
def analyze(self, file_path: str, tracer: PathTracer) -> CoverageReport:
    """ 分析覆盖率

    Args:
        file_path: 要分析的文件路径
        tracer: 已执行追踪的 PathTracer 实例

    Returns:
        CoverageReport: 覆盖率报告对象
    """
    # 1. 静态分析：获取所有分支点
    all_branches = self.cfg_builder.build_from_file(file_path)

    # 2. 动态追踪：获取已执行行号
    executed_lines = tracer.get_executed_lines(file_path)

    # 3. 标记已执行的分支
    for branch in all_branches:
        if branch.line_no in executed_lines:
            branch.is_executed = True

    # 4. 计算统计指标
    total = len(all_branches)
    executed = sum(1 for b in all_branches if b.is_executed)

    # 5. 按类型统计
    by_type: Dict[BranchType, int] = {}
    for branch_type in BranchType:
```

```

type_branches = [b for b in all_branches
                  if b.branch_type == branch_type]
by_type[branch_type] = len(type_branches)

# 6. 构建报告对象
return CoverageReport(
    total_branches=total,
    executed_branches=executed,
    branches_by_type=by_type,
    coverage_percentage=(executed / total * 100) if total > 0 else 0,
    file_reports={file_path: ExecutionPath(...)})

```

2.5.2 覆盖率计算公式

覆盖率计算公式：

$$\text{Coverage} = \frac{N_{executed}}{N_{total}} \times 100\% \quad (2-1)$$

其中：

- N_{total} ：静态分析识别的总分支数
- $N_{executed}$ ：运行时实际执行的分支数

2.5.3 多格式报告输出

表 2-3 报告输出格式

格式	方法	特点
文本	format_report()	纯文本，适合控制台输出
JSON	format_report_json()	结构化数据，便于程序处理
HTML	format_report_html()	带语法高亮，适合可视化查看

2.6 HTML 报告生成

2.6.1 HTML 报告结构

HTML 报告包含以下部分：

- 1) **摘要卡片**：覆盖率百分比、总分支数、已执行分支数
- 2) **分支类型统计**：按 if/for/while/except 分类的统计
- 3) **源代码展示**：带行号和执行状态高亮的代码
- 4) **函数调用树**：嵌套展示函数调用关系

2.6.2 执行状态视觉区分

表 2-4 代码行状态样式

状态	背景色	边框
已执行	绿色半透明 (rgba(40,167,69,0.15))	绿色左边框
未执行	红色半透明 (rgba(220,53,69,0.15))	红色左边框

2.7 本章小结

本章详细介绍了 PathTracer 系统的设计与实现：

- 1) **数据模型**：使用 Python dataclass 定义分支类型、代码分支、函数调用、覆盖率报告等核心数据结构
- 2) **运行时追踪**：基于 `sys.settrace` 实现，支持 line/call/return 事件追踪
- 3) **静态分析**：基于 AST 遍历识别 if/for/while/except 控制流结构
- 4) **报告生成**：支持文本/JSON/HTML 三种输出格式

整个系统仅依赖 Python 标准库，无需安装第三方依赖，具有良好的可移植性。

第三章 实验过程与结果

3.1 实验环境

表 3-1 实验环境配置

项目	配置
操作系统	Windows 11 Pro
Python 版本	3.10+
依赖	无第三方依赖（使用标准库）
对比工具	coverage.py 7.x

3.2 测试目标程序

3.2.1 程序简介

本实验使用 `agent_pageindex.py` 作为测试目标程序。该程序是一个基于 LangChain 的文档问答代理，共 130 行代码，具有以下特点：

- 多层条件判断（参数验证、类型检查、消息处理）
- 循环结构（消息处理、工具调用遍历）
- 函数调用（包含嵌套调用和递归调用树）
- 异常处理（API 密钥检查、异常捕获）

3.2.2 程序结构分析

目标程序包含以下主要函数：

表 3-2 目标程序结构

函数名	行号	功能描述
<code>_extract_text()</code>	24-37	从复杂消息结构中提取纯文本内容，支持 <code>None</code> 、 <code>str</code> 、 <code>list</code> 、 <code>dict</code> 等多种类型
<code>_print_verbose_stream()</code>	40-75	以流式输出方式打印 <code>Agent</code> 执行过程，包含工具调用、返回值、消息类型判断等复杂逻辑
<code>main()</code>	77-130	主函数，包含命令行参数解析、环境配置、 <code>Agent</code> 创建和执行

3.3 测试用例设计与执行

3.3.1 测试用例设计

为了全面验证工具功能，设计了以下三个测试用例：

表 3-3 测试用例设计

编号	测试内容	命令参数	预期覆盖
TC1	基本查询功能	<code>-query "test"</code>	主函数入口、参数解析、 <code>Agent</code> 执行
TC2	详细输出模式	<code>-query "test" -verbose</code>	TC1 + <code>_print_verbose_stream</code> 函数、消息处理
TC3	自定义参数	<code>-query "test" -doc-top-k 5</code>	TC1 + 参数传递分支

3.3.2 测试执行过程

TC1 执行命令：

```
python -m pathtracer.cli -f json -o coverage_tc1.json \
    agent_pageindex.py -- --query "test"
```

TC2 执行命令（verbose 模式）：

```
python -m pathtracer.cli -f json -o coverage_tc2.json \
    agent_pageindex.py -- --query "test" --verbose
```

TC3 执行命令（自定义参数）：

```
python -m pathtracer.cli -f json -o coverage_tc3.json \
    agent_pageindex.py -- --query "test" --doc-top-k 5
```

3.4 实验结果

3.4.1 覆盖率统计汇总

三个测试用例的覆盖率统计结果汇总：

表 3-4 测试用例覆盖率对比

用例	总分支	已执行	覆盖率	if 分支	for 分支
TC1	22	7	31.8%	5/17	2/5
TC2 (verbose)	22	16	72.7%	12/17	4/5
TC3	22	7	31.8%	5/17	2/5

关键发现：TC2 (verbose 模式) 覆盖率从 31.8% 显著提升至 72.7%，说明 verbose 模式触发了 `_print_verbose_stream` 函数中的大量消息处理逻辑。

3.4.2 测试套件累计覆盖率

将 TC1、TC2、TC3 三个测试用例的执行结果合并，计算测试套件的累计覆盖率：

表 3-5 测试套件累计覆盖率分析

测试用例	新增分支	累计覆盖	新增覆盖说明
TC1	7	7/22 (31.8%)	基础路径：main 入口、参数解析、Agent 执行
TC2	+9	16/22 (72.7%)	verbose 流式处理、消息遍历、工具调用追踪
TC3	+0	16/22 (72.7%)	无新增，验证参数传递路径正确性
累计	16	72.7%	–

分析：TC1 和 TC3 覆盖的分支完全重叠（都是非 verbose 路径），TC2 独有覆盖 9 个分支（全部来自 `_print_verbose_stream` 函数）。测试套件最终累计覆盖率为 72.7%。

3.4.3 TC2 详细分支覆盖分析

TC2 测试用例执行后，覆盖了 16 个分支，以下是详细列表：

表 3-6 TC2 已执行分支详情

行号	类型	代码内容	触发条件
25	if	if payload is None	空值检查
27	if	if isinstance(payload, str)	字符串类型检查
43	for	for chunk in agent.stream(...)	流式输出循环
44	if	if not isinstance(chunk, dict)	类型过滤
46	for	for step_name, payload in chunk.items()	字典遍历
47	if	if not isinstance(payload, dict)	类型过滤
50	for	for message in messages	消息遍历
52	if	if message_id in seen_messages	消息去重
57	if	if tool_calls	工具调用判断
58	for	for call in tool_calls	工具调用遍历
64	if	if not content	空内容过滤
67	if	if message_type == "ToolMessage"	工具消息判断
73	if	if message_type == "AIMessage"	AI 消息判断
88	if	if not api_key	API 密钥检查
116	if	if args.verbose	verbose 模式判断
129	if	if __name__ == "__main__"	入口判断

3.4.4 未覆盖分支分类分析

TC2 测试后仍有 6 个分支未覆盖（覆盖率 72.7%），按未覆盖原因分类：

表 3-7 未覆盖分支分类及原因

行号 未覆盖原因	类型	代码内容	分类
26 payload 为 None 时才执行，正常输入不为空	return	return ""	防御性路径
29 当前输入为 str 而非 list	if	if isinstance(payload, list)	输入类型相关
32 未进入 list 处理分支	if	if isinstance(item, str)	输入类型相关
34 未进入 list 处理分支	if	elif isinstance(item, dict)	输入类型相关
89 API 密钥已正确设置	raise	raise ValueError(...)	环境相关
126 messages 非空时走 124 行	print	print(_extract_text(result))	逻辑互斥

未覆盖分支分析：

- 防御性路径（2 个）：空值检查、异常抛出等保护性代码，正常流程难以触发
- 输入类型相关（3 个）：list 类型的 payload 处理分支，需要特定类型输入
- 逻辑互斥（1 个）：126 行与 124 行逻辑互斥，无法同时覆盖

提升建议：设计专门的 list 类型 payload 测试用例可覆盖第 29、32、34 行分支。

3.5 Demo 程序完整测试

3.5.1 Demo 程序结构

为了更全面地展示工具能力，对 pathtracer/demo.py 进行完整测试。该程序包含 251 行代码，涵盖：

- 选择结构：多层 if/elif/else 条件判断
- 循环结构：for 循环、while 循环

- 异常处理: try/except/else/finally
- 嵌套结构: 循环内嵌套条件、条件内嵌套循环
- 函数调用: 多层函数调用链

3.5.2 Demo 程序覆盖率结果

表 3-8 Demo 程序覆盖率统计

指标	数值
总分支数	14
已执行分支数	10
覆盖率	71.4%

3.5.3 分支类型分布

表 3-9 Demo 程序分支类型分布

分支类型	总数	已执行	覆盖率
if 语句	7	5	71.4%
for 循环	4	3	75.0%
while 循环	2	1	50.0%
except 块	1	1	100%

3.5.4 函数调用树

实验记录的完整函数调用关系:

Listing 3.1 Demo 程序函数调用树

```
demo_conditionals(value=25) -> 'medium'
demo_conditionals(value=-5) -> 'negative'
demo_conditionals(value=0) -> 'zero'
demo_for_loop(items=[1, 2, 3, 4, 5]) -> 15
demo_while_loop(limit=5) -> [0, 2, 4, 6, 8]
demo_exception_handling(dividend=10, divisor=2) -> 5.0
demo_exception_handling(dividend=10, divisor=0) -> 0.0
demo_nested_structures(data=[3, 7, 2], threshold=5) -> [...]
demo_function_calls(name='test', count=3) -> {...}
```

```

demo_conditionals(value=3) -> 'small'
demo_conditionals(value=30) -> 'medium'
demo_conditionals(value=-5) -> 'negative'
demo_for_loop(items=[0, 1, 2]) -> 3
demo_while_loop(limit=3) -> [0, 2, 4]
demo_exception_handling(dividend=100, divisor=3) -> 33.33...
demo_exception_handling(dividend=100, divisor=0) -> 0.0
demo_nested_structures(data=[5, 10, 15], threshold=3) -> [...]

```

3.6 与 coverage.py 对比分析

3.6.1 对比口径说明

重要说明：PathTracer 和 coverage.py 的覆盖率统计口径不同，必须明确定义以进行公平对比：

表 3-10 覆盖率统计口径对比

维度	PathTracer	coverage.py
统计对象	AST 控制流节点 (if/for/while/except)	源代码行号
分母定义	静态分析识别的分支点总数	可执行语句总数
分子定义	运行时实际执行的分支点数	运行时实际执行的语句数
分支模式	默认开启（核心功能）	需要显式指定 <code>--branch</code>
运行命令	<code>python -m pathtracer.cli demo.py</code>	<code>coverage run --branch -m demo</code>

3.6.2 对比实验设计

使用 Python 社区广泛使用的 `coverage.py` 工具对同一程序进行测试，对比分析：

Listing 3.2 coverage.py 对比测试

```

# 运行 coverage.py （开启 branch 模式以公平对比）
coverage run --branch -m pathtracer.demo
coverage report -m
coverage json -o coverage_py_demo.json

```

3.6.3 覆盖率对比结果

表 3-11 PathTracer vs coverage.py 覆盖率对比 (demo.py)

工具	语句覆盖	分支覆盖	分支数	函数调用追踪
PathTracer	–	71.4%	10/14	支持 (完整调用树)
coverage.py	52%	50%	2/4 (BrPart)	不支持

说明：coverage.py 的 --branch 模式统计的是布尔表达式跳转（如 if condition 的 true/false），而 PathTracer 统计的是控制流结构入口（进入 if 块、进入 for 循环）。两者口径不同，PathTracer 的 71.4% 与 coverage.py 的 50% 不可直接比较数值大小。

3.6.4 功能对比分析

表 3-12 功能特性对比

功能特性	PathTracer	coverage.py
语句覆盖	支持 (行号记录)	支持
分支覆盖	支持 (分支点识别)	部分支持
函数调用树	完整支持	不支持
参数/返回值记录	支持	不支持
静态分析	AST 解析	不支持
外部依赖	无	无
HTML 报告	基础	更丰富

3.6.5 对比结论

- 1) 分支覆盖：PathTracer 专注于分支级别的覆盖率分析 (71.4%)，而 coverage.py 主要提供语句覆盖 (57%)
- 2) 函数调用追踪：PathTracer 独有功能，能够记录完整的函数调用树、参数和返回值
- 3) 静态分析：PathTracer 使用 AST 静态识别所有分支点，与运行时追踪结合计算覆盖率
- 4) 轻量级：两者都仅依赖 Python 标准库

3.7 HTML 报告展示

PathTracer 生成的 HTML 报告包含以下特性 (完整报告见附录)：

- 响应式布局，支持各种屏幕尺寸

- 源代码语法高亮
- 已执行行（绿色）和未执行行（红色）视觉区分
- 覆盖率统计卡片展示
- 函数调用树可视化

3.8 结果分析与讨论

3.8.1 覆盖率分析结论

- 1) **verbose 模式影响显著**：覆盖率从 31.8% 提升至 72.7%，说明测试用例设计对覆盖率影响巨大
- 2) **未覆盖分支多为防御性代码**：如 API 密钥检查、空值处理等，这些代码在正常流程中难以触发
- 3) **嵌套结构覆盖完整**：demo.py 中的嵌套循环和条件组合均被正确追踪
- 4) **函数调用树准确**：多层嵌套调用的参数和返回值都被正确记录

3.8.2 工具优势

- 1) 无第三方依赖，纯标准库实现
- 2) 同时支持静态分析和动态追踪
- 3) 独有的函数调用树功能
- 4) 多格式报告输出

3.8.3 工具局限性

表 3-13 工具局限性及影响

局限性	具体影响	适用场景建议
性能开销（10-100 倍）	追踪回调在每行执行时都被调用，程序变慢	教学分析、调试阶段，不适合生产常驻
多线程支持不完整	追踪函数是线程局部的，跨线程路径可能丢失	单线程程序或主线程逻辑分析
exec/eval 不支持	动态生成的代码没有源文件关联	避免在追踪目标中使用动态代码生成
分支类型有限	不支持 with 语句、async/await、match-case	当前仅适用于传统控制流分析

3.8.4 覆盖率合理性说明

实验结果显示 `demo.py` 覆盖率为 71.4%，`agent_pageindex.py` 最高为 72.7%。这个覆盖率是合理的，原因如下：

- 1) **防御性代码占比较高**：未覆盖的 6 个分支中，有 2 个是空值检查和异常抛出，属于防御性编程
- 2) **输入类型相关分支**：2 个分支依赖特定的 `list` 类型输入，当前测试用例使用 `str` 类型
- 3) **逻辑互斥**：1 个分支与另一分支逻辑互斥（126 行与 124 行）

因此，72.7% 的覆盖率反映的是“有效业务路径已较充分覆盖”，而非测试设计不足。

3.8.5 实验对象说明

本实验主要使用 `demo.py` 和 `agent_pageindex.py` 作为测试对象，原因如下：

- 1) `demo.py` 是专门设计的测试程序，包含所有控制流结构（`if/for/while/except`/嵌套）
- 2) `agent_pageindex.py` 是真实项目代码，用于验证工具在复杂场景下的表现

局限性：本实验未对大规模项目（如 Django、Flask 框架）进行测试，因此工具在大规模代码库上的表现有待进一步验证。

3.9 HTML 报告与可视化

3.9.1 HTML 报告特性

PathTracer 生成的 HTML 报告（`coverage_demo_full.html`）包含以下可视化元素：

表 3-14 HTML 报告可视化元素

元素	说明
覆盖率卡片	大字号显示覆盖率百分比、已执行分支数、总分支数
分支类型统计	按类型显示分支数量（ <code>if/for/while/except</code> ）
源代码展示	深色背景、行号、语法高亮、执行状态着色
已执行行	绿色背景（ <code>rgba(40,167,69,0.15)</code> ）、绿色左边框
未执行行	红色背景（ <code>rgba(220,53,69,0.15)</code> ）、红色左边框
函数调用树	缩进显示函数名、参数、返回值

3.9.2 控制流图示例

PathTracer 识别的控制流结构示例（`demo.py demo_conditionals` 函数）：

Listing 3.3 控制流结构示例

```
def demo_conditionals(value: int) -> str:  
    if value < 0:                # Branch 1: IF (line 32)  
        return "negative"  
    elif value == 0:            # Branch 2: IF (line 34)  
        return "zero"  
    elif value <= 10:           # Branch 3: IF (line 36)  
        return "small"  
    elif value <= 50:           # Branch 4: IF (line 38)  
        return "medium"  
    else:                        # (implicit else)  
        return "large"
```

该函数包含 4 个 if 分支点，AST 静态分析可识别全部 4 个分支，运行时追踪可记录实际执行的分支。

3.10 本章小结

本章完成了完整的实验验证，包括：

- 1) 设计并执行了 3 个测试用例 (TC1/TC2/TC3)，测试套件累计覆盖率达 72.7%
- 2) 对 demo.py 进行了完整测试，覆盖了 if/for/while/except 等多种控制流结构
- 3) 与 coverage.py 进行了对比分析（使用-branch 模式），明确了两者统计口径差异
- 4) 对未覆盖分支进行了分类分析（防御性、输入类型、逻辑互斥）
- 5) 分析了工具的局限性及对应影响，说明了适用场景
- 6) 生成了 HTML 可视化报告，支持源代码着色和函数调用树展示

实验结论：PathTracer 能够有效分析 Python 程序的控制流结构，准确记录执行路径和函数调用关系，适用于教学演示、调试分析和代码审查场景。

第四章 使用文档

4.1 安装与配置

4.1.1 环境要求

- Python 3.10 或更高版本
- 无需安装第三方依赖，仅使用 Python 标准库

4.1.2 获取源码

将 `pathtracer` 目录复制到项目中即可使用。

4.2 命令行使用

4.2.1 基本用法

Listing 4.1 基本命令格式

```
python -m pathtracer.cli [选项] <程序文件> [-- <程序参数>]
```

4.2.2 命令行选项

表 4-1 命令行选项说明

选项	说明	默认值
<code>-f, --format</code>	报告格式 (text/html/json)	text
<code>-o, --output</code>	输出文件路径	stdout
<code>-q, --quiet</code>	静默模式，仅输出报告	否
<code>-</code>	分隔符，后接目标程序参数	-

4.2.3 使用示例

示例 1：生成文本格式报告

```
python -m pathtracer.cli agent_pageindex.py -- --query "test"
```

示例 2：生成 HTML 格式报告

```
python -m pathtracer.cli -f html -o report.html agent_pageindex.py -- --q
```

示例 3：静默模式生成 JSON 报告

```
python -m pathtracer.cli -f json -o report.json -q agent_pageindex.py --
```

4.3 API 使用

4.3.1 基本追踪流程

Listing 4.2 API 使用示例

```
from pathtracer import PathTracer, CoverageReporter

# 创建追踪器并指定目标文件
tracer = PathTracer().trace_file("target.py")

# 开始追踪
with tracer:
    # 在此执行目标代码
    result = target_function(args)

# 生成覆盖率报告
reporter = CoverageReporter()
report = reporter.analyze("target.py", tracer)

# 输出报告
print(reporter.format_report(report))

# 保存HTML报告
with open("report.html", "w") as f:
    f.write(reporter.format_report_html(report))
```

4.3.2 获取函数调用树

Listing 4.3 获取函数调用树

```
# 获取追踪到的函数调用
```

```
calls = tracer.get_function_calls()

def print_call_tree(call, depth=0):
    indent = "  " * depth
    args = ", ".join(f"{k}={v}" for k, v in call.arguments.items())
    ret = f"->{call.return_value}" if call.return_value else ""
    print(f"{indent}{call.function_name}({args}){ret}")
    for child in call.children:
        print_call_tree(child, depth + 1)

for call in calls:
    print_call_tree(call)
```

4.4 报告格式说明

4.4.1 文本格式

文本格式报告包含以下信息：

- 1) 总体覆盖率统计（总分支数、已执行分支数、覆盖率百分比）
- 2) 按分支类型统计
- 3) 已执行分支详情（行号、分支类型）

4.4.2 JSON 格式

JSON 格式报告结构：

Listing 4.4 JSON 报告结构

```
{
  "coverage_percentage": 31.8,
  "total_branches": 22,
  "executed_branches": 7,
  "branches_by_type": {
    "if": 17, "for": 5
  },
  "file_reports": {
```

```
"target.py": {  
    "executed_lines": [1, 2, 3],  
    "executed_branches": [...]  
}  
}
```

4.4.3 HTML 格式

HTML 格式报告特点:

- 响应式布局, 支持各种屏幕尺寸
- 语法高亮的源代码显示
- 已执行/未执行代码行视觉区分
- 函数调用树可视化

4.5 支持的程序结构

表 4-2 支持的程序结构

结构类型	说明
顺序结构	默认的线性执行流程
if/elif/else	条件分支语句
for 循环	迭代循环
while 循环	条件循环
try/except	异常处理块
函数调用	包含参数和返回值追踪
嵌套结构	循环内的条件语句等

4.6 本章小结

本章详细介绍了 PathTracer 工具的使用方法, 包括命令行接口和 Python API 两种使用方式。工具支持三种报告输出格式, 能够满足不同场景的需求。通过简单的配置即可对任意 Python 程序进行执行路径追踪和覆盖率分析。

结论

工作总结

本文设计并实现了一个 Python 程序执行路径追踪和代码覆盖率分析工具——Path-Tracer。该工具具有以下特点：

1. 技术实现

- 使用 Python 标准库 `sys.settrace` 实现运行时追踪，记录程序执行的每一行代码
- 使用 `ast` 模块进行静态分析，识别源代码中的所有控制流分支点
- 通过对比静态分支与动态执行路径，计算代码覆盖率

2. 功能特性

- 支持识别顺序、选择 (`if/elif/else`)、循环 (`for/while`)、异常处理 (`try/except`) 等程序结构
- 记录函数调用树，包含参数和返回值信息
- 支持文本、JSON、HTML 三种报告输出格式
- 提供命令行接口和 Python API 两种使用方式

3. 实验验证

通过完整的实验验证，工具成功：

- 设计并执行了 3 个测试用例 (TC1/TC2/TC3)，测试套件累计覆盖率达 72.7%
- 对 `demo.py` 进行了完整测试，覆盖了 `if/for/while/except` 等多种控制流结构
- 与 `coverage.py` 进行了对比分析 (使用 `-branch` 模式)，明确了两者统计口径差异
- 生成了 HTML 可视化报告 (见 `docs/coverage_demo_full.html`)

4. 工具定位

本实验验证了 PathTracer 在 Python 分支覆盖与执行路径跟踪上的可行性，适用于以下场景：

- **教学演示：**帮助理解程序控制流和执行路径
- **调试分析：**定位未覆盖的代码分支，辅助测试设计
- **代码审查：**记录函数调用关系和参数传递

局限性：本实验主要针对小型示例程序进行验证，尚未针对大型真实项目（如 Django、Flask 等框架）做系统评测。工具在大规模代码库上的性能和准确性有待进一步验证。

工作展望

未来可以从以下方面改进工具：

- 1) **提高覆盖率精度**：增加对 `elif` 分支、`else` 分支的独立识别，区分 `true/false` 两个执行方向
- 2) **支持更多语法**：扩展支持 `with` 语句、`async/await`、`match-case` 等 Python 3.10+ 新特性
- 3) **集成开发环境**：开发 IDE 插件，实现实时覆盖率显示
- 4) **增量分析**：支持增量覆盖率分析，只分析变更的代码
- 5) **测试用例关联**：将覆盖率与具体测试用例关联，便于定位测试盲点
- 6) **大规模验证**：在真实大型项目上进行系统评测，验证工具的泛化能力

参考文献

感谢张老师在课程设计过程中给予的悉心指导和帮助。

感谢同学们在开发过程中的讨论与建议，帮助完善了工具的功能设计。

感谢开源社区提供的 **Python** 标准库文档和示例，为本项目的开发提供了重要参考。

许同学

2026 年 4 月 2 日

附录 A 核心源代码

A.1 数据模型 (models.py)

核心数据模型定义见项目源文件 `pathtracer/models.py`。

A.2 运行时追踪器 (tracer.py)

运行时追踪器实现见项目源文件 `pathtracer/tracer.py`。

A.3 控制流图构建器 (cfg_builder.py)

控制流图构建器实现见项目源文件 `pathtracer/cfg_builder.py`。

A.4 报告生成器 (reporter.py)

报告生成器实现见项目源文件 `pathtracer/reporter.py`。

A.5 命令行接口 (cli.py)

命令行接口实现见项目源文件 `pathtracer/cli.py`。